AGENT ENGINEERING

Stop Prompting Agents. Start Designing Loops.

Most developers use coding agents like a faster search box. There is a better way.

10 slides • Practical system design • Real guardrails

THE CORE SHIFT

A prompt gives you an answer.

A loop gives you a workflow.

When you prompt an agent you ask: what should it do?

When you design a loop you ask:

- Where do tasks come from?
- Which decisions are safe to delegate?
- What proves the result is correct?
- Who makes the irreversible decision?

ANATOMY OF A LOOP

Five moves.
Every loop.

1. **Wake**
   on a schedule or event
2. **Read state**
   from a visible control plane
3. **Choose**
   one bounded, policy-allowed action
4. **Change**
   the world: update a ticket or open a PR
5. **Record**
   evidence and stop. Repeat later.

The loop is easy. Designing the boundaries around it is the real engineering.

CONTROL PLANE

Visible state beats agent memory.

GitHub Issues works as a control plane because it already has:

- Durable tickets and acceptance criteria
- Labels that act as a small policy language
- Comments that form an audit trail
- Links from issues to branches and pull requests
- Human permissions and override controls

Agents read from it before acting. They write evidence back after.

A human can reroute work by changing a label. No hunting for hidden prompts.

LABEL TAXONOMY

Labels are your policy language.

RISK

`risk:low`

`risk:medium`

`risk:high`

TYPE

`type:bug`

`type:docs`

`type:test`

`type:feature`

`type:refactor`

ROUTING

`agent-ready`

`needs-human`

`blocked`

```
gh issue list --search \
  'is:open label:agent-ready label:risk:low \
   -label:needs-human' --limit 1
```

The worker queue is a transparent, deterministic query.

ARCHITECTURE

Two loops.
Different jobs.

LOOP 1 — MANAGER

- Reads open issues
- Applies risk + routing labels
- Never touches code

permissions: issues:write contents:read

LOOP 2 — WORKER

- Reads safe queue only
- Writes code + tests, opens PR
- Never merges or deploys

permissions: contents:write pulls:write

The manager earns the worker. Neither earns the merge button.

WORKER INNER LOOP

Input: one ticket.

Output: one PR.

1. **Verify clean tree**
   abort if dirty
2. **Claim the issue**
   add agent-running label
3. **Implement + test**
   smallest complete change
4. **Fresh agent review**
   new context, no shared bias
5. **Fix findings**
   rerun checks once
6. **Open PR with evidence**
   exact commands, outputs, risks

The coordinator writes the worker prompt. The worker writes the review prompt.

GUARDRAILS

Prompts are not permissions.

"Never deploy" is a useful instruction. It is not a security control.

Real guardrails sit at multiple layers:

- Selection — reject any issue missing required labels
- Quantity — cap at 1 ticket until metrics hold, then 3 max
- Environment — abort on dirty working tree
- Permissions — least-privilege token, branch protection enforced
- Stop conditions — escalate on ambiguity, scope creep, or repeated failure

A branch protection rule beats a sentence in a prompt every time.

EVALUATION

You cannot increase trust without metrics.

First-pass merge rate
PRs merged with no requested changes

Rework rate
PRs substantially rewritten by a human

Escalation precision
Agent asks for help at the right time

Cost per merged PR
Tokens + CI spend per useful output

Policy violations
Any prohibited action attempted — target: 0

A weekly scorecard beats a story about one impressive pull request.

GET STARTED

Build one small loop this week.

- Create one clear issue with explicit acceptance criteria
- Apply risk:low and agent-ready labels
- Run the manager loop in dry-run mode first
- Allow one documentation or test PR before anything bigger
- Measure first-pass merge rate before expanding scope

Full handbook, scripts, and workflows:

`github.com/seunboy1/loop-engineering-handbook`